The Potential of Controversy Affecting Viewership Numbers in Modern Media

Niki Khajavand †
† Department of Mathematics, California State University, Fullerton

Abstract

One of the main indicators of success in the online entertainment industry is the amount of views a creator receives on their content. Although there are many different factors that can lead to an increase or decrease of average views, this project will be mainly focusing on controversy and seeing if there is an obvious change in viewership amount pre and post scandal. This study will be done by focusing on the YouTube group, The Try Guys, and their 2022 scandal with their previous member. Using the data collected from their videos between October 2021 and October 2023, we will see how their average viewership on their videos have changed between one year before the incident and one year after.

1 Introduction

In 2014, four employees of the online entertainment company, BuzzFeed Inc., formed a group called The Try Guys, and started to post content on the company's YouTube channel. Due to their popularity, the group was able to break away from their parent company in 2018, and was able to start producing Try Guys content under their own company, 2nd Try LLC (Business Insider, 2022). Like many others in the entertainment industry, the group was hit with controversy when it was revealed that one of the members was having an extramarital affair with one of their employees and was ultimately terminated from the group.

Since the trajectory of one's career in modern media, especially online entertainment, is heavily dependant on the amount of video views (Backlinko, 2017), the goal of this project is to see if the average amount of monthly views can be affected by other factors, like increased publicity from controversy, than the main sources of Youtube search and related video recommendations (Yang, 2022). Since the official announcement addressing the scandal happened in October 2022, I will be using that date as the midpoint for all of the data and I will be looking at the monthly average views starting from October 2021 until October 2023.

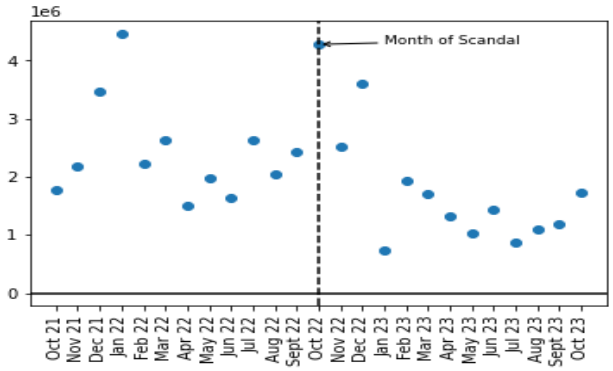

Figure 1: (a) One of the many ways fans have reacted to and brought attention to the scandal on the internet was crossing out the face of the former member on the old Try Guys YouTube header. (b) The display of average Try Guys video views per month from October 2021 to October 2023.

Table 1: The average amount of views on Try Guys YouTube videos each month starting from a year before the scandal up to a year after the scandal.

Month	Average Views	Month	Average Views
October 2021	1,764,096	November 2022	2,526,506
November 2021	2,189,474	December 2022	3,611,316
December 2021	3,463,682	January 2023	743,170
January 2022	4,465,010	February 2023	1,922,541
February 2022	2,218,769	March 2023	1,705,668
March 2022	2,626,130	April 2023	1,326,187
April 2022	1,505,382	May 2023	1,029,268
May 2022	1,979,853	June 2023	1,433,912
June 2022	1,636,089	July 2023	880,781
July 2022	2,636,005	August 2023	1,100,544
August 2022	2,055,372	September 2023	1,191,511
September 2022	2,439,880	October 2023	1,733,155
October 2022	4,278,833		

2 Methods

In order to form the data set, I calculated the average amount of views for all of their videos during each month during the two year time period between October of 2021 and October 2023. I did this by noting the current amount of views each video has from that time frame, segregated them by their months, then calculated the average for each month. It should be noted that the amount of views for each video taken into account has increased since the original data collection, so the current average amount of views might be slightly different than the ones currently recorded. In terms of both interpolation and integration, multiple methods will be used to help analyze the data gathered. The methods used to help interpret this data will be Newton's Divided Difference and Cubic Spline for interpolation and both the trapezoidal and Simpson's rule for integration.

In order to create the polynomial needed for the further integration techniques, I first used Newton's Divided Difference method in order to find the coefficients for Newton's Interpolated Polynomial. Since my original set of x-values were of each individual month, I assigned each month with a number between 0 and 24 that corresponds to the order they show up in, eg the first month of October 2021 being 0, the last month of October 2023 being 24, and October 2022 (the month of the scandal) being the midpoint at 12. With these assigned numbers, I created a new array of x-values that could be used in Python to help numerically find the data needed. With these x-values and the original y-values, I was able to construct the 25x26 table on Python, and from there, construct the polynomial found in Appendix

C. Since the main focus of this project was to compare average overall views pre and post scandal, I have also divided the data pre and post scandal and found their respective 12^{th} degree polynomials, also found in Appendix C.

Now that the polynomial was found in one way, I then used Cubic Spline Interpolation in order to find the interpolant in the form of a piecewise polynomial. By using Cubic Spline Interpolation, I am able to split the data into a series of cubic polynomials between pairs of nodes. One of the main benefits of of node splitting in cubic spline is that it allows a more cohesive graph to be shown that can be viewed all at once. The polynomial found by the divided differences table unfortunately has several big peaks, thus washing out the rest of the graph and making it difficult to analyze without separating it into multiple smaller graphs. It should be noted that even with scaling the data into smaller numbers, I obtain the same polynomials, and thus the same graphs as well. Since I have 25 data points, the splitting of data into pairs of nodes will give me 24 cubic polynomials. Going from intervals of x's between $[0, 1)$, $[1, 2)$, $[2, 3)$, $[3, 4)$, $[4, 5)$, $[5, 6)$, $[6, 7)$, $[7, 8)$, $[8, 9)$, $[9, 10)$, $[10, 11)$, $[11, 12)$, $[12, 13)$, $[13, 14)$, $[14, 15)$, $[15, 16)$, $[16, 17)$, $[17, 18)$, $[18, 19)$, $[19, 20)$, $[20, 21)$, $[21, 22)$, $[22, 23)$, and $[23, 24]$, I was able to construct the following Cubic Spline found in Appendix C along with corresponding graph below. By using $y = mx + b$, I have also found and plotted the Linear Spline for comparison.

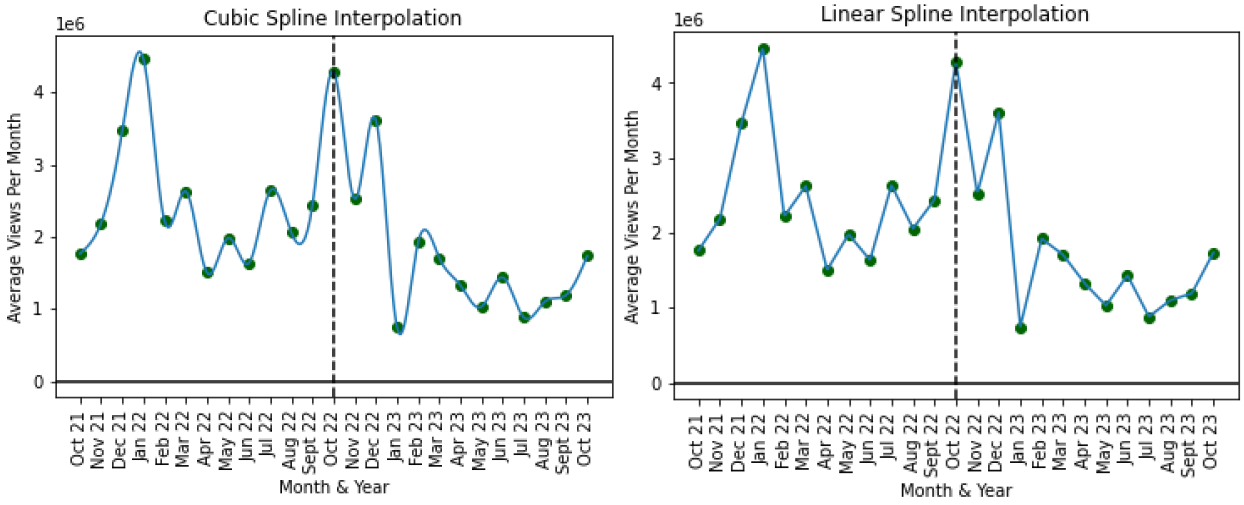

Figure 2: (a) Cubic Spline Interpolation between data points. (b) Linear Spline Interpolation between data points.

Although both splines are almost visually identical, cubic spline is preferred over linear spline due to smoother curves leading to a smaller margin of error. One notable instance of this is the data point for January 2022 being one of the peaks of the linear spline graph, whereas the cubic spline showing that it is not truly at the apex. The data was once again split pre and post scandal, and I was able to calculate and plot individual spline graphs for both groups of piecewise functions. All spline equations can also be found in Appendix C.

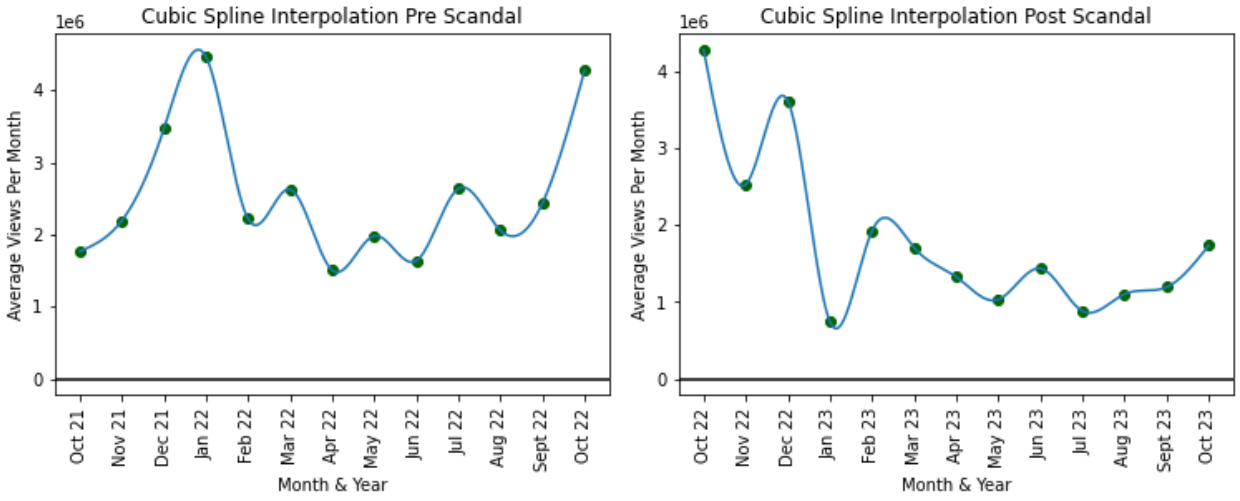

Figure 3: (a) Cubic Spline Interpolation between data points pre scandal. (b) Cubic Spline Interpolation between data points post scandal.

Since we want to see a comparison of average views pre and post scandal, it is best to find a trend line for the overall data, and this can be done by performing a regression analysis. I first used the Newton's Interpolating Polynomial found earlier to find a polynomial regression since I initially want to find a nonlinear regression due to each observation being assumed to be independent of each other. This can be done by using the newly assigned x-values to be the independent variables, the y-values being the dependent variables, and the coefficients found in the polynomial be the coefficients/parameters of the polynomial regression equation. If starting from scratch, I can otherwise find the polynomial using the least squares method and find the best fit line using the R-squared method.

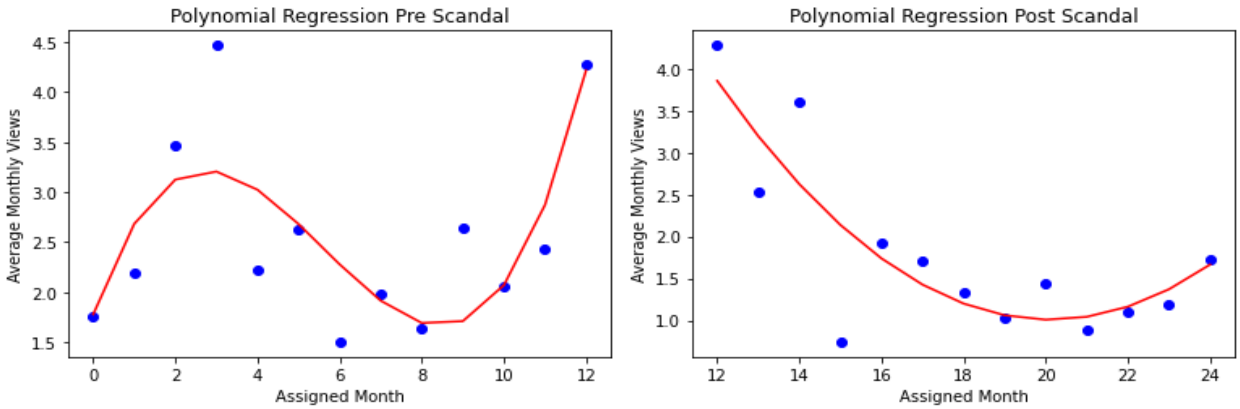

Figure 4: (a) Polynomial Regression for October 2021 to October 2022. (b) Polynomial Regression for October 2022 to October 2023.

However, one of the main the issues with polynomial regression is that overfitting

can occur due to a higher degree polynomial order, and that happened to be the case here. Although the polynomials used were of twelfth degree, the twelfth degree regression was overfit and did not best represent the data. I had to scale the degree down to better match the data, and found that using a cubic regression pre scandal (Appendix C) and a quadratic regression post scandal (Appendix C) found the most accurate line of best fit. Another issue with polynomial regression is that it is very sensitive to outliers, which ultimately poses an issue with my data since it has multiple outliers, both pre and post scandal. This is evident with the presence of corners in my cubic regression, as each corner in what should be a smooth curve corresponds to each data point that is an outlier. Other issues that polynomial regressions pose are that it could potentially not perform on newer data points if we wanted to predict the trend line for future dates, especially with higher degree polynomials, and that it can be difficult to interpret. Due to these reasons, I have also decided to find a linear regression in order to have a more obvious trend line. I did this by using $y = mx + b$ in order to find the line of best fit, with the actual equation found in Appendix C, and applied it to both the pre and post scandal data.

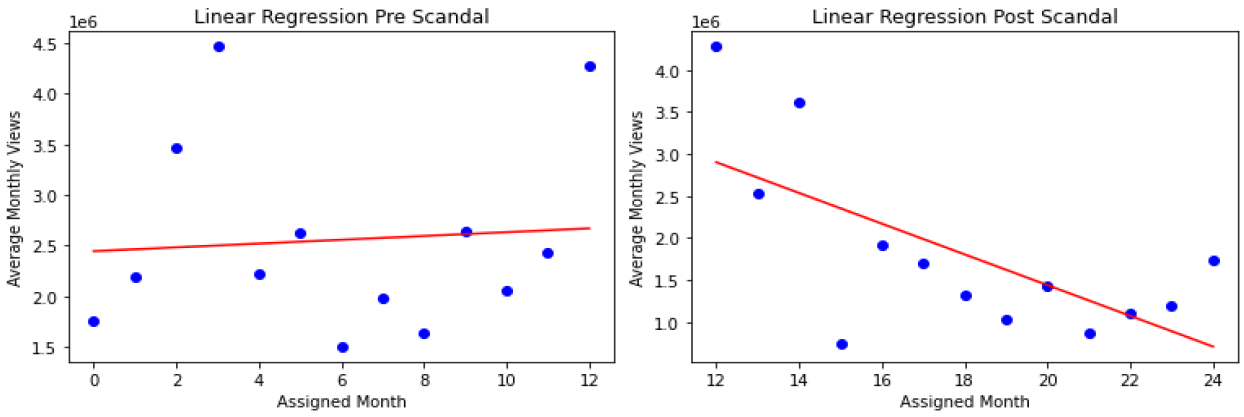

Figure 5: (a) Linear Regression for October 2021 to October 2022. (b) Linear Regression for October 2022 to October 2023.

3 Numerical Results

Although the linear regression graphs show a variance of slopes that are indicative of a decrease of overall viewership, I wanted to see if using integration techniques would also lead to the same results. By implementing these techniques, I can find the numerical values of the definite integrals associated with the functions found for both splines and both regressions. In order to have better precision with the integral approximation and because my data is spaced out in increments of 1, I used a small integral range from 0 to 1 and spliced them into evenly spaced nodes that increased in intervals of 2^i. I applied those bounds and n's to the piecewise linear spline polynomials, the piecewise cubic spline polynomials, the cubic regression polynomial, the quadratic regression polynomial, and the linear regression

equations to the Composite Trapezoidal Rule twelve consecutive times. However, this rule does not guarantee convergence, so I then applied the same parameters and equations to the Composite Simpson's Rule, which is evidently more accurate since there is overall a smaller remainder. This allowed me to get the area under the curve, with each increasing iteration of n making the values more similar and precise. By finding the numerical value for the area under the curve, I can compare the pre scandal and post scandal areas and see if there were any major changes.

Table 2: The average amount of views on Try Guys YouTube videos each month starting from a year before the scandal up to a year after the scandal.

	Pre Scandal	Post Scandal
Linear Spline	30,237,110.5	20,477,398
Cubic Spline	30,081,881.28	20,179,589.12
Linear Regression	30,700,223.08	21,676,977.23
Polynomial Regression	30,221,917.49	20,636,650.69

For each instance pre scandal that the Composite Simpson's Rule was applied to, it was found that the area has consistently fallen into the range between 30,000,000 and 31,000,000. Conversely, for each instance post scandal that the Composite Simpson's Rule was applied to, it was found that the area has consistently fallen into the range between 20,000,000 and 22,000,000. Each instance has shown the area to decrease by a third after the scandal has happened, thus corroborating the data found with the regressions.

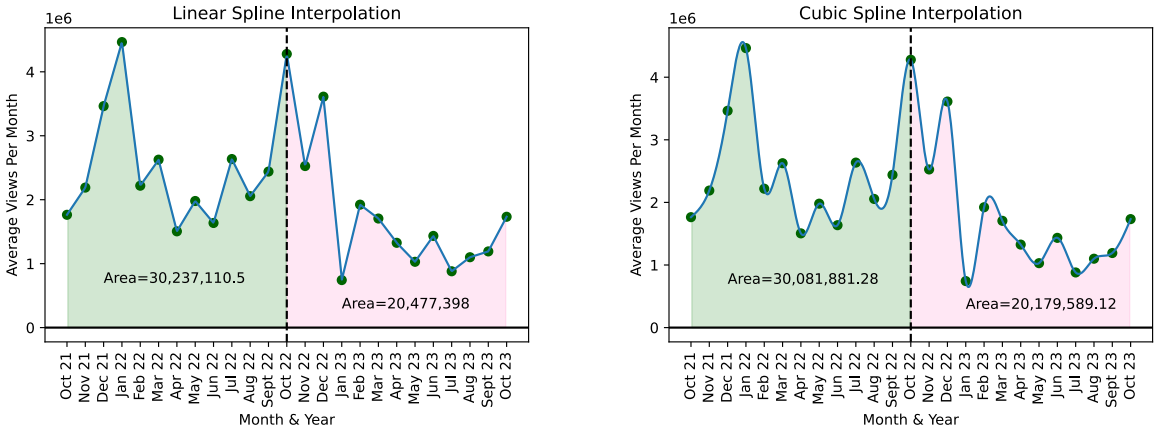

Figure 6: (a) Comparative areas under the curve found by the Composite Simpson's Rule for pre and post scandal linear spline. (b) Comparative areas under the curve found by the Composite Simpson's Rule for pre and post scandal cubic spline.

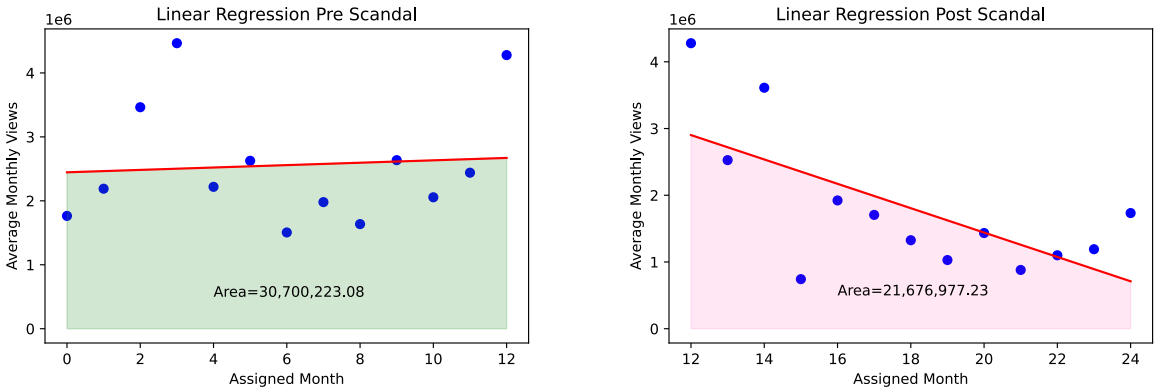

Figure 7: Comparative areas under the curve found by the Composite Simpson's Rule for pre and post scandal linear regression.

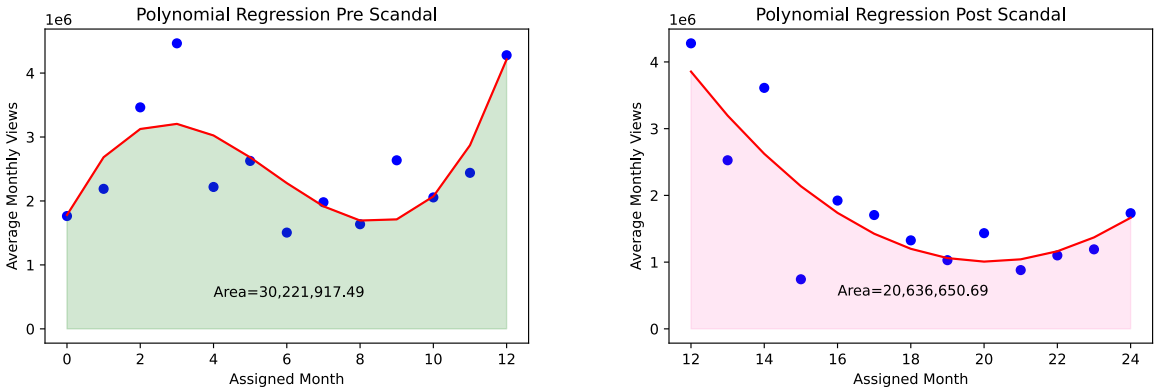

Figure 8: Comparative areas under the curve found by the Composite Simpson's Rule for pre and post scandal polynomial regression.

To further numerically simulate the data, I have used the logistic differential equation $\frac{dP}{dt} rP(1 - \frac{P}{K})$ with initial condition $P(0) = P_0$. With t and P being variables that correlate to time and population, r being the growth rate, K being the carrying capacity, and P_0 being the initial population, this model usually tends to simulate the change of a population density over time. I have decided to use this model since the documentation of change of online viewership/presence before and after a scandal mimics the change of overall population before and after a catastrophic event. The generalized answer to this equation tends to follow a sigmoid curve, and I wanted to compare this curve with the linear regression graphs found earlier.

Since the data point with the largest amount of average views was January 2022 with 4,465,010 views, I used that as my K for the function. By using Runge-Kutta of order 4 to approximate my data, I found that the result for the initial-value problem

$$\begin{cases} p' = -0.15p(1 - \frac{p}{4{,}465{,}010}) \\ p(0) = 4,000,000 \end{cases}$$

best suited the overall data. After experimenting with different growth rates, I was able to find that having $r = -0.15$ allowed the sigmoid curve to best follow the post scandal trendline. If using this model to further predict the amount of average monthly views after October 2023, it would suggest that the amount of views would stabilize towards the average of around 1,000,000 views per month.

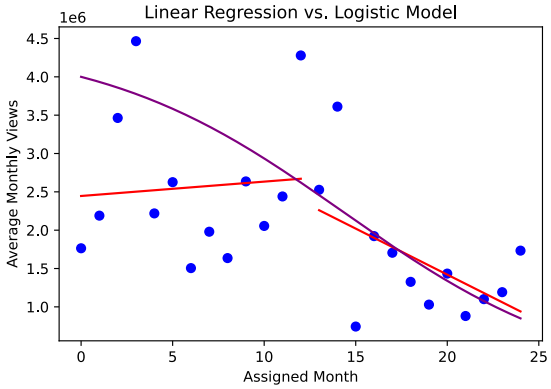

Figure 9: The comparative graphs for Linear Regression and the predicted Logistic Model.

4 Conclusions

Given the trend lines found from the linear regression analysis, it is shown that pre scandal there is a positive slope, and post scandal there is a negative slope. The positive slope expects a predicted increase of average views, and the negative slope expects a predicted decrease of average views, thus it can be inferred that the scandal has had some negative effect on the average amount of views The Try Guys have received on their videos since there has been a decreased number of average monthly video views after October 2022. This can be better seen with the overall linear regression of both years, which shows a clear decreasing trend line.

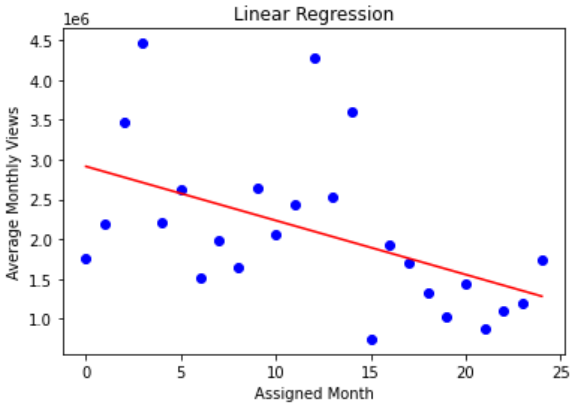

Figure 10: Linear Regression for October 2021 to October 2023.

Furthermore, when comparing the areas found from integration techniques, it was found that the amount of average monthly views has decreased by almost a third post scandal for each instance calculated. This was found by calculating the norm for the area pre and post scandal and finding the percent difference between them, as shown below.

$$\text{linear spline percent difference} = 100(1 - \frac{(\sum_{i=12}^{23} \int_i^{i+1} S(x))/12}{(\sum_{i=0}^{11} \int_i^{i+1} S(x))/12}) \approx 32.277265712939074 \approx 32\%$$

$$\text{cubic spline percent difference} = 100(1 - \frac{(\sum_{i=12}^{23} \int_i^{i+1} S(x))/12}{(\sum_{i=0}^{11} \int_i^{i+1} S(x))/12}) \approx 32.917795492343616 \approx 33\%$$

$$\text{linear regression percent difference} = 100(1 - \frac{(\int_{12}^{24} P(x))/12}{(\int_0^{12} P(x))/12}) \approx 29.39146672155061 \approx 29\%$$

$$\text{polynomial regression percent difference} = 100(1 - \frac{(\int_{12}^{24} P_2(x))/12}{(\int_0^{12} P_3(x))/12}) \approx 31.71627612037399 \approx 32\%$$

However, correlation does not imply causation and there are many other factors that could contribute to the change of average views. One notable factor is the fact that there has been a different number of videos outputted each month, thus affecting actual average number of views that each month has and causing outliers in the data. For example, December of 2021 had 12 videos released that much versus January of 2023, which only had two, thus causing the former to have a much higher view average compared to the rest of the data than the latter, which happens to have a much lower view average compared to the rest of the data. Another factor is that some months will have videos pertaining to a popular series that The Try Guys release annually, thus causing the average amount of views for that month to potentially be higher since there is more anticipation for those particular videos. Although the data does show that the average amount of video views per month has decreased since the scandal, it cannot be implied that it is the sole reason for the decreased average viewership.

References

[1] The Try Guys. (2022) 27 September. Available at: `https://www.instagram.com/p/CjBSYpfp5Om/?igsh=Y2RkOTNiMDgwYQ==` [Accessed: 5 March 2024].

[2] Business Insider. (2022). *The Try Guys built a hugely successful YouTube channel as uncontroversial nice guys. Then explosive cheating rumors changed everything.* [online]. Available from: `https://www.businessinsider.com/history-the-try-guys-buzzfeed-ned-fulmers-departure-cheating-2022-9` [Accessed: 28 March 2024].

[3] Yang, S., Brossard, D., Scheufele, D., Xenos, M. (2022). 'The science of YouTube: What factors influence user engagement with online science videos?'. *PLOS ONE* [online]. 17(5). Available from: `https://doi.org/10.1371/journal.pone.0267697` [Accessed: 29 March 2024].

Appendix A Dataset and Figure Construction

A.1. The Try Guys. (2024). *The Try Guys YouTube Channel Video Section* [online]. Available from: `https://www.youtube.com/@tryguys/videos` [Accessed: 5 March 2024].

A.2. Pink Tea Latte. (2022). *Ned Fulmer Fired from The Try Guys for Cheating is October's Biggest Internet Scandal* [online]. Available from: `https://www.pinktealatte.ca/2022/10/ned-fulmer-fired-from-try-guys-for.html` [Accessed: 28 March 2024].

Appendix B Code

```
# importing libraries
import matplotlib.pyplot as plt
import numpy as np
import sympy as sym

# creating two array for plotting
X = ['Oct 21', 'Nov 21', 'Dec 21', 'Jan 22', 'Feb 22', 'Mar 22', 'Apr 22',
     'May 22', 'Jun 22', 'Jul 22', 'Aug 22', 'Sept 22', 'Oct 22', 'Nov 22',
     'Dec 22', 'Jan 23', 'Feb 23', 'Mar 23', 'Apr 23', 'May 23', 'Jun 23',
     'Jul 23', 'Aug 23', 'Sept 23', 'Oct 23']
Y = [1764096, 2189474, 3463682, 4465010, 2218769, 2626130, 1505382, 1979853,
     1636089, 2636005, 2055372, 2439880, 4278833, 2526506, 3611316, 743170,
     1922541, 1705668, 1326187, 1029268, 1433912, 880781, 1100544, 1191511, 1733155]

# creating scatter plot with both negative
# and positive axes
plt.scatter(X, Y)
plt.xticks(rotation=90)

# adding vertical line in data co-ordinates
plt.axvline('Oct 22', c='black', ls='--')

# adding horizontal line in data co-ordinates
plt.axhline(0, c='black', ls='-')

# visualizing the plot using plt.show() function
plt.annotate('Month of Scandal', xy=(12,4278833), xytext=(15,4278833),
             fontsize=9, arrowprops=dict(arrowstyle='->'))
plt.savefig("RawData.svg")
plt.show()

#%% DD
xs = [0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24]
```

```
x=sym.symbols('x')
def myDD(X,y):
    n=len(X)
    DD=np.zeros([n,n+1])
    DD[:,0]=X
    DD[[range(0,n)],1]=y
    for j in range(0,n-1):
        DD[j,2]=(DD[j+1,1]-DD[j,1])/(DD[j+1,0]-DD[j,0])
    for j in range(3,n+1):
        for i in range(0,(n+1)-j):
            DD[i,j]=(DD[i+1,j-1]-DD[i,j-1])/(DD[i+(j-1),0]-DD[i,0])
    for i in range(0,n):
        for j in range(0,n+1):
            if i+j>n:
                DD[i,j]=np.nan
    NIP=0
    for i in range(1,n+1):
        prod=DD[0,i]
        for j in range(0,i-1):
            prod=prod*(x-X[j])
        NIP=NIP+prod
    p=sym.simplify(NIP)
    return(p)

poly=myDD(xs,Y)
print(poly)

#%% split DD

X1 = np.array([0,1,2,3,4,5,6,7,8,9,10,11,12])
Y1 = [1764096, 2189474, 3463682, 4465010, 2218769, 2626130, 1505382, 1979853,
     1636089, 2636005, 2055372, 2439880, 4278833]

X2 = np.array([12,13,14,15,16,17,18,19,20,21,22,23,24])
Y2 = [4278833, 2526506, 3611316, 743170,
     1922541, 1705668, 1326187, 1029268, 1433912, 880781, 1100544, 1191511, 1733155]

poly1=myDD(X1,Y1)
print(poly1)
poly2=myDD(X2,Y2)
print(poly2)

#%% Cubic Spline
X1 = np.array([0,1,2,3,4,5,6,7,8,9,10,11,12])
Y1 = [1764096, 2189474, 3463682, 4465010, 2218769, 2626130, 1505382, 1979853,
```

```
     1636089, 2636005, 2055372, 2439880, 4278833]

X2 = np.array([12,13,14,15,16,17,18,19,20,21,22,23,24])
Y2 = [4278833, 2526506, 3611316, 743170,
     1922541, 1705668, 1326187, 1029268, 1433912, 880781, 1100544, 1191511, 1733155]

def myCubicSpline(xs,ys):
    n=len(xs)
    x,a,b,c,d=sym.symbols('x a b c d')
    a=sym.symbols('a:%d'%(n-1))
    b=sym.symbols('b:%d'%(n-1))
    c=sym.symbols('c:%d'%(n-1))
    d=sym.symbols('d:%d'%(n-1))

    sn=np.array([])
    for i in range(0,n-1):
        sn=np.append(sn,a[i]+b[i]*x+c[i]*x**2+d[i]*x**3)

    equations=np.array([])
    for i in range(0,n-1):
        #evaluating sj at t_j and t_j+1
        sjtj=(sn[i]-ys[i]).subs(x,xs[i])
        sjtj1=(sn[i]-ys[i+1]).subs(x,xs[i+1])
        if i<(n-2):
            #smoothness
            smooth=sym.diff(sn[i],x,1).subs(x,xs[i+1])-sym.diff(sn[i+1],x,1).subs(x,xs[i
            #double smoothness
            doublesmooth=sym.diff(sn[i],x,2).subs(x,xs[i+1])-sym.diff(sn[i+1],x,2).subs(
        equations=np.append(equations,[sjtj,sjtj1,smooth,doublesmooth])
    #natural conditions
    equations=np.append(equations,sym.diff(sn[0],x,2).subs(x,xs[0]))
    equations=np.append(equations,sym.diff(sn[-1],x,2).subs(x,xs[-1]))

    coeffs=sym.solve(equations,dict=True)
    sol_array=np.array([])
    for i in range(0,n-1):
        sol_array=np.append(sol_array,coeffs[0][a[i]]+coeffs[0][b[i]]*x+coeffs[0][c[i]]*
    return(sol_array)

spline_funcs=myCubicSpline(xs,Y)
spline_funcs1=myCubicSpline(X1,Y1)
spline_funcs2=myCubicSpline(X2,Y2)
print(sym.simplify(spline_funcs1))
print(sym.simplify(spline_funcs2))
#%% plotting cubic spline
```

```
x=sym.symbols('x')
func=0*x
func=sym.Piecewise((func,False),(spline_funcs[0],x<=xs[1]))

for i in range(1,len(xs)-2):
    func=sym.Piecewise((func,x<=xs[i]),(spline_funcs[i],x<=xs[i]+1))

func=sym.Piecewise((func,x<=xs[-1]-1),(spline_funcs[-1],x>xs[-1]-1))

x_new=np.linspace(0,24,500)
y_new=[func.subs(x,Y) for Y in x_new]

plt.plot(x_new,y_new)
plt.xticks(rotation=90)
plt.axhline(0, c='black', ls='-')
plt.scatter(X, Y, c='darkgreen')
plt.axvline('Oct 22', c='black', ls='--')
plt.title('Cubic Spline Interpolation')
plt.xlabel('Month & Year')
plt.ylabel('Average Views Per Month')
plt.savefig("CubicSpline.svg")
plt.show()
#%% plotting filled cubic spline

x=sym.symbols('x')
func=0*x
func=sym.Piecewise((func,False),(spline_funcs[0],x<=xs[1]))

for i in range(1,len(xs)-2):
    func=sym.Piecewise((func,x<=xs[i]),(spline_funcs[i],x<=xs[i]+1))

func=sym.Piecewise((func,x<=xs[-1]-1),(spline_funcs[-1],x>xs[-1]-1))

x_new=np.linspace(0,24,500)
y_new=[func.subs(x,Y) for Y in x_new]

plt.plot(x_new,y_new)
plt.fill_between(x_new.astype(np.float64),np.asarray(y_new).astype(np.float64),
                 where=(0<x_new)&(x_new<12),color='forestgreen',alpha=0.2)
plt.fill_between(x_new.astype(np.float64),np.asarray(y_new).astype(np.float64),
                 where=(12<x_new)&(x_new<24),color='deeppink',alpha=0.1)
plt.annotate('Area=30,081,881.28',xy=(2,700000),xytext=(2,700000),fontsize=10)
plt.annotate('Area=20,179,589.12',xy=(15,300000),xytext=(15,300000),fontsize=10)
plt.xticks(rotation=90)
```

```
plt.axhline(0, c='black', ls='-')
plt.scatter(X, Y, c='darkgreen')
plt.axvline('Oct 22', c='black', ls='--')
plt.title('Cubic Spline Interpolation')
plt.xlabel('Month & Year')
plt.ylabel('Average Views Per Month')
plt.savefig("FilledCubicSpline.svg")
plt.show()
#%% plotting cubic spline pre

x1 = ['Oct 21', 'Nov 21', 'Dec 21', 'Jan 22', 'Feb 22', 'Mar 22', 'Apr 22',
     'May 22', 'Jun 22', 'Jul 22', 'Aug 22', 'Sept 22', 'Oct 22']

x=sym.symbols('x')
func=0*x
func=sym.Piecewise((func,False),(spline_funcs1[0],x<=X1[1]))

for i in range(1,len(X1)-2):
    func=sym.Piecewise((func,x<=X1[i]),(spline_funcs1[i],x<=X1[i]+1))

func=sym.Piecewise((func,x<=X1[-1]-1),(spline_funcs1[-1],x>X1[-1]-1))

x_new=np.linspace(0,12,500)
y_new=[func.subs(x,Y1) for Y1 in x_new]

plt.plot(x_new,y_new)
plt.xticks(rotation=90)
plt.axhline(0, c='black', ls='-')
plt.scatter(x1, Y1, c='darkgreen')
plt.title('Cubic Spline Interpolation Pre Scandal')
plt.xlabel('Month & Year')
plt.ylabel('Average Views Per Month')
plt.savefig("PreCubicSpline.svg")
plt.show()
#%% plotting cubic spline post

x2 = ['Oct 22', 'Nov 22', 'Dec 22', 'Jan 23', 'Feb 23', 'Mar 23', 'Apr 23',
     'May 23', 'Jun 23', 'Jul 23', 'Aug 23', 'Sept 23', 'Oct 23']

x=sym.symbols('x')
func=0*x
func=sym.Piecewise((func,False),(spline_funcs2[0],x<=X2[1]))

for i in range(1,len(X2)-2):
    func=sym.Piecewise((func,x<=X2[i]),(spline_funcs2[i],x<=X2[i]+1))
```

```
func=sym.Piecewise((func,x<=X2[-1]-1),(spline_funcs2[-1],x>X2[-1]-1))

x_new1=np.linspace(12,24,500)
y_new1=[func.subs(x,Y2) for Y2 in x_new1]

plt.plot(x_new1,y_new1)
plt.xticks(X2, x2, rotation=90)
plt.axhline(0, c='black', ls='-')
plt.scatter(X2, Y2, c='darkgreen')
plt.title('Cubic Spline Interpolation Post Scandal')
plt.xlabel('Month & Year')
plt.ylabel('Average Views Per Month')
plt.savefig("PostCubicSpline.svg")
plt.show()

#%% linear spline

x,m,b=sym.symbols('x m b')
y=m*x+b
n=len(xs)-1
sx=np.array([])

for i in range(0,n):
    slope=(y-Y[i]).subs(x,xs[i])
    yint=(y-Y[i+1]).subs(x,xs[i+1])
    sol=sym.solve([slope,yint],[m,b])
    sx=np.append(sx,sol[m]*x+sol[b])

print(sx)

xs_new=np.linspace(0,24,500)
ys_new=np.interp(xs_new, xs, Y)
plt.plot(xs_new,ys_new)
plt.xticks(rotation=90)
plt.axhline(0, c='black', ls='-')
plt.scatter(X, Y, c='darkgreen')
plt.axvline('Oct 22', c='black', ls='--')
plt.title('Linear Spline Interpolation')
plt.xlabel('Month & Year')
plt.ylabel('Average Views Per Month')
plt.savefig("LinearSpline.svg")
plt.show()
#%% filled linear spline
```

```
x,m,b=sym.symbols('x m b')
y=m*x+b
n=len(xs)-1
sx=np.array([])

for i in range(0,n):
    slope=(y-Y[i]).subs(x,xs[i])
    yint=(y-Y[i+1]).subs(x,xs[i+1])
    sol=sym.solve([slope,yint],[m,b])
    sx=np.append(sx,sol[m]*x+sol[b])

print(sx)

xs_new=np.linspace(0,24,500)
ys_new=np.interp(xs_new, xs, Y)
plt.plot(xs_new,ys_new)
plt.fill_between(xs_new.astype(np.float64),np.asarray(ys_new).astype(np.float64),
                 where=(0<x_new)&(x_new<12),color='forestgreen',alpha=0.2)
plt.fill_between(xs_new.astype(np.float64),np.asarray(ys_new).astype(np.float64),
                 where=(12<x_new)&(x_new<24),color='deeppink',alpha=0.1)
plt.annotate('Area=30,237,110.5',xy=(2,700000),xytext=(2,700000),fontsize=10)
plt.annotate('Area=20,477,398',xy=(15,300000),xytext=(15,300000),fontsize=10)
plt.xticks(rotation=90)
plt.axhline(0, c='black', ls='-')
plt.scatter(X, Y, c='darkgreen')
plt.axvline('Oct 22', c='black', ls='--')
plt.title('Linear Spline Interpolation')
plt.xlabel('Month & Year')
plt.ylabel('Average Views Per Month')
plt.savefig("FilledLinearSpline.svg")
plt.show()

#%% CSR and CTR

def myCSR(f,a,b,n):
    x=sym.symbols('x')
    if n%2!=0:
        print('must be even, try again')
    chop=np.linspace(a,b,n+1)
    h=(b-a)/n
    even=0
    odd=0
    if n%2==0:
        for i in range(2,int(n/2)+1,1):
            even=even+f.subs(x,chop[2*i-2])
```

```
        for i in range(1,int(n/2)+1,1):
            odd=odd+f.subs(x,chop[2*i-1])
    csr_sum=(h/3)*(f.subs(x,a)+2*even+4*odd+f.subs(x,b))
    return csr_sum

def myCTR(f,a,b,n):
    x=sym.symbols('x')
    chop=np.linspace(a,b,n+1)
    h=(b-a)/n
    ctr_sum=0
    sol=f.subs(x,a)+f.subs(x,b)
    for i in range(1,n):
        ctr_sum=ctr_sum+f.subs(x,chop[i])
    sol=sol+2*ctr_sum
    sol=(h/2)*sol
    return sol

#%% split data pre cubic spline CSR

x=sym.symbols('x')
a=11
b=12
print('n Composite Simpsons   Composite Trapezoidal')
for i in range(1,10):
    n=2**i
    CSR=myCSR(spline_funcs1[11], a, b, n)
    print(n,CSR)

#%% split data post cubic spline CSR

x=sym.symbols('x')
a=23
b=24
print('n Composite Simpsons')
for i in range(1,10):
    n=2**i
    CSR=myCSR(spline_funcs2[11], a, b, n)
    print(n,CSR)

#%% cubic spline percent difference

pre_norm=30081881.28/12
post_norm=20179589.12/12
diff=1-(post_norm/pre_norm)
print(diff)
```

```
#%% split data pre linear spline CSR

x=sym.symbols('x')
a=11
b=12
print('n Composite Simpsons')
for i in range(1,2):
    n=2**i
    CSR=myCSR(sx[11], a, b, n)
    print(n,CSR)

#%% split data post linear spline CSR

x=sym.symbols('x')
a=23
b=24
print('n Composite Simpsons')
for i in range(1,2):
    n=2**i
    CSR=myCSR(sx[23], a, b, n)
    print(n,CSR)

#%% linear spline percent difference

pre_norm=30237110.5/12
post_norm=20477398/12
diff=1-(post_norm/pre_norm)
print(diff)
#%% linear regression and polynomial regression

xs = np.array([0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24]).reshap
Y = [1764096, 2189474, 3463682, 4465010, 2218769, 2626130, 1505382, 1979853,
     1636089, 2636005, 2055372, 2439880, 4278833, 2526506, 3611316, 743170,
     1922541, 1705668, 1326187, 1029268, 1433912, 880781, 1100544, 1191511, 1733155]


from sklearn.linear_model import LinearRegression
lin = LinearRegression()

lin.fit(xs, Y)

from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=24)
X_poly = poly.fit_transform(xs)
```

```
poly.fit(X_poly, Y)
lin2 = LinearRegression()
lin2.fit(X_poly, Y)

plt.scatter(xs, Y, color='blue')

plt.plot(xs, lin.predict(xs), color='red')
plt.title('Linear Regression')
plt.xlabel('Assigned Month')
plt.ylabel('Average Monthly Views')
plt.savefig("LinearRegression.svg")

plt.show()

plt.scatter(xs, Y, color='blue')

plt.plot(xs, lin2.predict(poly.fit_transform(xs)),
         color='red')
plt.title('Polynomial Regression')
plt.xlabel('Assigned Month')
plt.ylabel('Average Monthly Views')
plt.savefig("PolynomialRegression.svg")

plt.show()

#%% regression split into two parts

X1 = np.array([0,1,2,3,4,5,6,7,8,9,10,11,12]).reshape((-1, 1))
Y1 = [1764096, 2189474, 3463682, 4465010, 2218769, 2626130, 1505382, 1979853,
     1636089, 2636005, 2055372, 2439880, 4278833]

X2 = np.array([12,13,14,15,16,17,18,19,20,21,22,23,24]).reshape((-1, 1))
Y2 = [4278833, 2526506, 3611316, 743170, 1922541, 1705668, 1326187, 1029268,
      1433912, 880781, 1100544, 1191511, 1733155]

from sklearn.linear_model import LinearRegression
lin = LinearRegression()

lin.fit(X1, Y1)

from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=3)
X_poly = poly.fit_transform(X1)
```

```
poly.fit(X_poly, Y1)
lin2 = LinearRegression()
lin2.fit(X_poly, Y1)

plt.scatter(X1, Y1, color='blue')

plt.plot(X1, lin.predict(X1), color='red')
plt.title('Linear Regression Pre Scandal')
plt.xlabel('Assigned Month')
plt.ylabel('Average Monthly Views')
plt.savefig("PreLinearRegression.svg")

plt.show()
print(f"intercept: {lin.intercept_}")
print(f"slope: {lin.coef_}")

plt.scatter(X1, Y1, color='blue')

plt.plot(X1, lin2.predict(poly.fit_transform(X1)),
         color='red')
plt.title('Polynomial Regression Pre Scandal')
plt.xlabel('Assigned Month')
plt.ylabel('Average Monthly Views')
plt.savefig("PrePolynomialRegression.svg")

plt.show()
print(f"intercept: {lin2.intercept_}")
print(f"slope: {lin2.coef_}")

from sklearn.linear_model import LinearRegression
lin3 = LinearRegression()

lin3.fit(X2, Y2)

from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X2)

poly.fit(X_poly, Y2)
lin4 = LinearRegression()
lin4.fit(X_poly, Y2)

plt.scatter(X2, Y2, color='blue')
```

```
plt.plot(X2, lin3.predict(X2), color='red')
plt.title('Linear Regression Post Scandal')
plt.xlabel('Assigned Month')
plt.ylabel('Average Monthly Views')
plt.savefig("PostLinearRegression.svg")

plt.show()
print(f"intercept: {lin3.intercept_}")
print(f"slope: {lin3.coef_}")

plt.scatter(X2, Y2, color='blue')

plt.plot(X2, lin4.predict(poly.fit_transform(X2)),
         color='red')
plt.title('Polynomial Regression Post Scandal')
plt.xlabel('Assigned Month')
plt.ylabel('Average Monthly Views')
plt.savefig("PostPolynomialRegression.svg")

plt.show()
print(f"intercept: {lin4.intercept_}")
print(f"slope: {lin4.coef_}")
#%% regression split into two parts (filled)

X1 = np.array([0,1,2,3,4,5,6,7,8,9,10,11,12]).reshape((-1, 1))
Y1 = [1764096, 2189474, 3463682, 4465010, 2218769, 2626130, 1505382, 1979853,
     1636089, 2636005, 2055372, 2439880, 4278833]

X2 = np.array([12,13,14,15,16,17,18,19,20,21,22,23,24]).reshape((-1, 1))
Y2 = [4278833, 2526506, 3611316, 743170, 1922541, 1705668, 1326187, 1029268,
      1433912, 880781, 1100544, 1191511, 1733155]

from sklearn.linear_model import LinearRegression
lin = LinearRegression()

lin.fit(X1, Y1)

from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=3)
X_poly = poly.fit_transform(X1)

poly.fit(X_poly, Y1)
lin2 = LinearRegression()
```

```
lin2.fit(X_poly, Y1)

plt.scatter(X1, Y1, color='blue')

plt.plot(X1, lin.predict(X1), color='red')
plt.title('Linear Regression Pre Scandal')
plt.xlabel('Assigned Month')
plt.ylabel('Average Monthly Views')
plt.fill_between(np.array([0,1,2,3,4,5,6,7,8,9,10,11,12]),lin.predict(X1),
                 0,color='forestgreen',alpha=0.2)
plt.annotate('Area=30,700,223.08',xy=(4,500000),xytext=(4,500000),fontsize=10)
plt.savefig("FilledPreLinearRegression.svg")

plt.show()
print(f"intercept: {lin.intercept_}")
print(f"slope: {lin.coef_}")

plt.scatter(X1, Y1, color='blue')

plt.plot(X1, lin2.predict(poly.fit_transform(X1)),
         color='red')
plt.title('Polynomial Regression Pre Scandal')
plt.xlabel('Assigned Month')
plt.ylabel('Average Monthly Views')
plt.fill_between(np.array([0,1,2,3,4,5,6,7,8,9,10,11,12]),
                 lin2.predict(poly.fit_transform(X1)),0,color='forestgreen',alpha=0.2)
plt.annotate('Area=30,221,917.49',xy=(4,500000),xytext=(4,500000),fontsize=10)
plt.savefig("FilledPrePolynomialRegression.svg")

plt.show()
print(f"intercept: {lin2.intercept_}")
print(f"slope: {lin2.coef_}")

from sklearn.linear_model import LinearRegression
lin3 = LinearRegression()

lin3.fit(X2, Y2)

from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X2)

poly.fit(X_poly, Y2)
lin4 = LinearRegression()
```

```
lin4.fit(X_poly, Y2)

plt.scatter(X2, Y2, color='blue')

plt.plot(X2, lin3.predict(X2), color='red')
plt.title('Linear Regression Post Scandal')
plt.xlabel('Assigned Month')
plt.ylabel('Average Monthly Views')
plt.fill_between(np.array([12,13,14,15,16,17,18,19,20,21,22,23,24]),lin3.predict(X2),
                 0,color='deeppink',alpha=0.1)
plt.annotate('Area=21,676,977.23',xy=(16,500000),xytext=(16,500000),fontsize=10)
plt.savefig("FilledPostLinearRegression.svg")

plt.show()
print(f"intercept: {lin3.intercept_}")
print(f"slope: {lin3.coef_}")

plt.scatter(X2, Y2, color='blue')

plt.plot(X2, lin4.predict(poly.fit_transform(X2)),
         color='red')
plt.title('Polynomial Regression Post Scandal')
plt.xlabel('Assigned Month')
plt.ylabel('Average Monthly Views')
plt.fill_between(np.array([12,13,14,15,16,17,18,19,20,21,22,23,24]),
                 lin4.predict(poly.fit_transform(X2)),0,color='deeppink',alpha=0.1)
plt.annotate('Area=20,636,650.69',xy=(16,500000),xytext=(16,500000),fontsize=10)
plt.savefig("FilledPostPolynomialRegression.svg")

plt.show()
print(f"intercept: {lin4.intercept_}")
print(f"slope: {lin4.coef_}")
#%% split data pre linear regression CSR

x=sym.symbols('x')
f=18728.35164835*x+2445981.813186813
a=0
b=12
print('n Composite Simpsons')
for i in range(1,2):
    n=2**i
    CSR=myCSR(f, a, b, n)
    print(n,CSR)

#%% split data post linear regresion CSR
```

```
x=sym.symbols('x')
f=-182598.65934066*x+5093190.6373626385
a=12
b=24
print('n Composite Simpsons')
for i in range(1,2):
    n=2**i
    CSR=myCSR(f, a, b, n)
    print(n,CSR)

#%% linear regression percent difference

pre_norm=30700223.08/12
post_norm=21676977.23/12
diff=1-(post_norm/pre_norm)
print(diff)
#%% split data pre poly regression (cubic) CSR

x=sym.symbols('x')
f=1780431.0082417484+1167936.16978859*x-281160.85389611*x**2+16727.23630536*x**3
a=0
b=12
print('n Composite Simpsons')
for i in range(1,2):
    n=2**i
    CSR=myCSR(f, a, b, n)
    print(n,CSR)

#%% split data post poly regresion (quadratic) CSR

x=sym.symbols('x')
f=18530741.746253766-1743088.46553446*x+43346.93906094*x**2
a=12
b=24
print('n Composite Simpsons')
for i in range(1,2):
    n=2**i
    CSR=myCSR(f, a, b, n)
    print(n,CSR)

#%% poly regression percent difference

pre_norm=30221917.49/12
post_norm=20636650.69/12
```

```
diff=1-(post_norm/pre_norm)
print(diff)

#%% logistic model w/ RKMO4 for pre

X1 = np.array([0,1,2,3,4,5,6,7,8,9,10,11,12])
Y1 = [1764096, 2189474, 3463682, 4465010, 2218769, 2626130, 1505382, 1979853,
     1636089, 2636005, 2055372, 2439880, 4278833]

X2 = np.array([12,13,14,15,16,17,18,19,20,21,22,23,24])
Y2 = [4278833, 2526506, 3611316, 743170,
     1922541, 1705668, 1326187, 1029268, 1433912, 880781, 1100544, 1191511, 1733155]

# dP/dt=-rP(1-P/K)(1-P/T), P(0)=P_0
# K is carrying capacity (max of data)
# T is threshold population (min of data)
# need to test around different r's

#dP/dt=rp(1-P/K), P(0)=P_0

f1=lambda t,p: -0.15*p*(1-p/4465010)
a=0
b=24
h=0.1
N=int((b-a)/h)
t1=np.linspace(a,b,N+1)
x0=4000000
w1=np.array([x0])

for i in range(0,N):
    k1=h*f1(t1[i],w1[i])
    k2=h*f1(t1[i]+h/2,w1[i]+k1/2)
    k3=h*f1(t1[i]+h/2,w1[i]+k2/2)
    k4=h*f1(t1[i]+h,w1[i]+k3)
    w1=np.append(w1,w1[i]+(1/6)*(k1+2*k2+2*k3+k4))


X1 = np.array([0,1,2,3,4,5,6,7,8,9,10,11,12]).reshape((-1, 1))
Y1 = [1764096, 2189474, 3463682, 4465010, 2218769, 2626130, 1505382, 1979853,
     1636089, 2636005, 2055372, 2439880, 4278833]

X2 = np.array([13,14,15,16,17,18,19,20,21,22,23,24]).reshape((-1, 1))
Y2 = [2526506, 3611316, 743170, 1922541, 1705668, 1326187, 1029268,
      1433912, 880781, 1100544, 1191511, 1733155]
```

```
from sklearn.linear_model import LinearRegression
lin = LinearRegression()

lin.fit(X1, Y1)

from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=3)
X_poly = poly.fit_transform(X1)

poly.fit(X_poly, Y1)
lin2 = LinearRegression()
lin2.fit(X_poly, Y1)

lin3 = LinearRegression()

lin3.fit(X2, Y2)

from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X2)

poly.fit(X_poly, Y2)
lin4 = LinearRegression()
lin4.fit(X_poly, Y2)

plt.scatter(xs, Y, color='blue')

plt.plot(X1, lin.predict(X1), color='red')
#plt.plot(X2, lin3.predict(X2), color='red')
plt.title('Linear Regression vs. Logistic Model')
plt.xlabel('Assigned Month')
plt.ylabel('Average Monthly Views')


plt.plot(X2, lin3.predict(X2), color='red')

plt.plot(t1,w1,color='purple')
plt.savefig("RegressionVsLogisticModel.svg")
plt.show()
```

Appendix C Equations

C.1. Newton's Interpolated Polynomial found by the Divided Differences method:

$$\begin{aligned}P_{24}(x) =& 1.20331977982739e^{-11}x^{24} - 3.44146299614394e^{-9}x^{23} + 4.63791093538724e^{-7}x^{22} \\ & - 3.91626875537752e^{-5}x^{21} + 0.00232408056956917x^{20} - 0.103054499395553x^{19} \\ & + 3.54391550031935x^{18} - 96.8279755225029x^{17} + 2136.20114683488x^{16} \\ & - 38466.2234821153x^{15} + 569192.619267009x^{14} - 6946326.06809371x^{13} \\ & + 69977620.0103879x^{12} - 580975683.855207x^{11} + 3958364895.87352x^{10} \\ & - 21976670892.0893x^{9} + 98391741322.3218x^{8} - 350023337205.568x^{7} \\ & + 969180226467.566x^{6} - 2028256945639.93x^{5} + 3071715643916.32x^{4} \\ & - 3141869275101.59x^{3} + 1915354605234.04x^{2} - 515956515998.248x + 1764096.0\end{aligned}$$

C.2. Pre Scandal Newton's Interpolated Polynomial found by the Divided Differences method:

$$\begin{aligned}P_{12}(x) =& - 3.33252730888582x^{12} + 241.550336036456x^{11} - 7732.28931370793x^{10} + \\ & 143889.89298184x^{9} - 1723764.28841649x^{8} + 13901965.666715x^{7} - \\ & 76632938.4262699x^{6} + 286896923.591187x^{5} - 708953233.53676x^{4} + \\ & 1089017712.22501x^{3} - 921675629.126713x^{2} + 319457946.073774x + 1764096.0\end{aligned}$$

C.3. Post Scandal Newton's Interpolated Polynomial found by the Divided Differences method:

$$\begin{aligned}P_{12}(x) =& 1.32751338408891x^{12} - 288.072480146204x^{11} + 28541.1514951912x^{10} - \\ & 1707111.18856027x^{9} + 68649759.3181165x^{8} - 1955306185.70003x^{7} + \\ & 40444150051.1516x^{6} - 612093893565.163x^{5} + 6726637893329.79x^{4} - \\ & 52345767352687.0x^{3} + 273784881829356.0x^{2} - 864107930143063.0x + \\ & 1.24450339316058e + 15\end{aligned}$$

C.4. Piecewise polynomial found by Cubic Spline Interpolation:

$$S(x) = \begin{cases} S_0(x) = \frac{198905631762291943}{1183606020240}x^3 + \frac{304574329915358777}{1183606020240}x + 176409 \\ S_1(x) = \frac{2030427869771897}{236721204048}x^3 + \frac{94376746206716229}{197267670040}x^2 - \frac{261686147324938597}{1183606020240}x + \frac{379458023049122583}{197267670040} \\ S_2(x) = -\frac{1567176898121140283}{1183606020240}x^3 + \frac{238815111953816571}{28181095720}x^2 - \frac{19189634596964935813}{1183606020240}x + \frac{2482563406342455607}{197267670040} \\ S_3(x) = \frac{2737695644393996287}{1183606020240}x^3 - \frac{2392801515047994429}{98633835020}x^2 + \frac{97041924050943751577}{1183606020240}x - \frac{8444681517487829479}{98633835020} \\ S_4(x) = -\frac{479788831477828765}{236721204048}x^3 + \frac{2743838286735145683}{98633835020}x^2 - \frac{149516786434646973799}{1183606020240}x + \frac{3790146151737783557}{19726767004} \\ S_5(x) = \frac{1908582670658748373}{1183606020240}x^3 - \frac{5281140496649439129}{197267670040}x^2 + \frac{173547725668944941051}{1183606020240}x - \frac{10367736146723983711}{39453534008} \\ S_6(x) = -\frac{1538596701261690947}{1183606020240}x^3 + \frac{5060397619111878831}{197267670040}x^2 - \frac{198747646498462505509}{1183606020240}x + \frac{2064565047300454199}{5636219144} \\ S_7(x) = \frac{277845090083141291}{236721204048}x^3 - \frac{1296744977939753019}{49316917510}x^2 + \frac{46328441959622982517}{236721204048}x - \frac{3396930583882451947}{7045273930} \\ S_8(x) = -\frac{1459449491153975273}{1183606020240}x^3 + \frac{1551929963629928709}{49316917510}x^2 - \frac{315303378983263979191}{1183606020240}x + \frac{1056949085704077521}{1409054786} \\ S_9(x) = \frac{987437465239799677}{1183606020240}x^3 - \frac{4803271449252272439}{197267670040}x^2 + \frac{39898593060060476237}{169086574320}x - \frac{29864778640654560697}{39453534008} \\ S_{10}(x) = \frac{104558727971908433}{236721204048}x^3 - \frac{25567549714958607}{2033687320}x^2 + \frac{139897003806346080059}{1183606020240}x - \frac{14376651127979310297}{39453534008} \\ S_{11}(x) = -\frac{2499468864550455377}{1183606020240}x^3 + \frac{7071195725952000801}{98633835020}x^2 - \frac{1635852599557773603353}{1183606020240}x + \frac{299277654960860618291}{98633835020} \\ S_{12}(x) = \frac{3502931331866805343}{1183606020240}x^3 - \frac{10936004863299781359}{98633835020}x^2 + \frac{1635852599557773603353}{1183606020240}x - \frac{565067973323224925389}{98633835020} \\ S_{13}(x) = -\frac{780708680220721183}{236721204048}x^3 + \frac{26270076037708110459}{197267670040}x^2 - \frac{302747155722603557779}{169086574320}x + \frac{225981174010887962599}{28181095720} \\ S_{14}(x) = \frac{4074447319758135997}{1183606020240}x^3 - \frac{5915171801664816585}{39453534008}x^2 + \frac{367404064829782762829}{169086574320}x - \frac{59049510617082501797}{5636219144} \\ S_{15}(x) = -\frac{2924837870361364553}{1183606020240}x^3 + \frac{572969497939304280}{4931691751}x^2 - \frac{2152689049522183531447}{1183606020240}x + \frac{46759126196108287412}{4931691751} \\ S_{16}(x) = \frac{236327173867919915}{236721204048}x^3 - \frac{1241626250004442728}{24658458755}x^2 + \frac{1001082782568156918857}{1183606020240}x - \frac{116623461473940835196}{24658458755} \\ S_{17}(x) = -\frac{341566610612241107}{1183606020240}x^3 + \frac{3014211079555103973}{197267670040}x^2 - \frac{319533767550088952437}{1183606020240}x + \frac{314261272209038863543}{197267670040} \\ S_{18}(x) = \frac{474815261091605653}{1183606020240}x^3 - \frac{4333225765779516867}{197267670040}x^2 + \frac{67712773106578585469}{169086574320}x - \frac{479261907087100187177}{197267670040} \\ S_{19}(x) = -\frac{165008224723920253}{236721204048}x^3 + \frac{4007704944488474427}{98633835020}x^2 - \frac{933755052896186993911}{1183606020240}x + \frac{14381294048122778875}{2818109572} \\ S_{20}(x) = \frac{861346786973798287}{1183606020240}x^3 - \frac{4424234608478523333}{98633835020}x^2 + \frac{1089910439815892468489}{1183606020240}x - \frac{17740380439370545925}{2818109572} \\ S_{21}(x) = -\frac{571915776832851323}{1183606020240}x^3 + \frac{6200787703012774239}{197267670040}x^2 - \frac{806295932100304965541}{1183606020240}x + \frac{27726118185132155951}{5636219144} \\ S_{22}(x) = \frac{71814121593480281}{236721204048}x^3 - \frac{4040062529790005769}{197267670040}x^2 + \frac{109099259725932399103}{236721204048}x - \frac{681776367745889716339}{197267670040} \\ S_{23}(x) = -\frac{178498923670220777}{1183606020240}x^3 + \frac{535496771010662331}{49316917510}x^2 - \frac{307626548079244407319}{1183606020240}x + \frac{20416252374663423253}{9863383502} \end{cases}$$

C.5. Pre scandal piecewise polynomial found by Cubic Spline Interpolation:

$$S(x) = \begin{cases} S_0 = \frac{354175153949}{2107560}x^3 + \frac{542334503731}{2107560}x + 1764096 \\ S_1 = \frac{3616877011}{421512}x^3 + \frac{168045384447}{351260}x^2 - \frac{465937802951}{2107560}x + \frac{675671489109}{351260} \\ S_2 = -\frac{2790583821769}{2107560}x^3 + \frac{425244798753}{50180}x^2 - \frac{34169956284839}{2107560}x + \frac{4420562431541}{351260} \\ S_3 = \frac{4874915353181}{2107560}x^3 - \frac{4260767585577}{175630}x^2 + \frac{172798521438811}{2107560}x - \frac{15037091927867}{175630} \\ S_4 = -\frac{854401127639}{421512}x^3 + \frac{4886153405799}{175630}x^2 - \frac{266253686147237}{2107560}x + \frac{6749297338561}{35126} \\ S_5 = \frac{3399900364439}{2107560}x^3 - \frac{9407458194987}{351260}x^2 + \frac{309139264050313}{2107560}x - \frac{18467680333853}{70252} \\ S_6 = -\frac{2744994659881}{2107560}x^3 + \frac{9027226877973}{351260}x^2 - \frac{354509398576247}{2107560}x + \frac{3682223405893}{10036} \\ S_7 = \frac{498715832569}{421512}x^3 - \frac{2326945375392}{87815}x^2 + \frac{83112190672895}{421512}x - \frac{6092642297366}{12545} \\ S_8 = -\frac{2672956414099}{2107560}x^3 + \frac{218430015504}{6755}x^2 - \frac{576413877408773}{2107560}x + \frac{1930597987426}{2509} \\ S_9 = \frac{2035258422311}{2107560}x^3 - \frac{9828605957637}{351260}x^2 + \frac{81097475405551}{301080}x - \frac{60352876876835}{70252} \\ S_{10} = -\frac{20576571749}{421512}x^3 + \frac{862100447643}{351260}x^2 - \frac{73760056477943}{2107560}x + \frac{10918499158365}{70252} \\ S_{11} = -\frac{592489449091}{2107560}x^3 + \frac{1777468347273}{175630}x^2 - \frac{251487248773541}{2107560}x + \frac{6277059913471}{13510} \end{cases}$$

C.6. Post scandal piecewise polynomial found by Cubic Spline Interpolation:

$$S(x) = \begin{cases} S_0 = \frac{237928110227}{210756}x^3 - \frac{713784330681}{17563}x^2 + \frac{102177702078625}{210756}x - \frac{2583019783790}{1351} \\ S_1 = -\frac{591696905563}{210756}x^3 + \frac{3964993941273}{35126}x^2 - \frac{45491740132415*}{30108}x + \frac{2600247387215}{386} \\ S_2 = \frac{697806671717}{210756}x^3 - \frac{5061531099687}{35126}x^2 + \frac{62826560359105}{30108}x - \frac{50444351015165}{5018} \\ S_3 = -\frac{513382093717}{210756}x^3 + \frac{2011192320534}{17563}x^2 - \frac{377766494154215}{210756}x + \frac{164091611725235}{17563} \\ S_4 = \frac{208416409835}{210756}x^3 - \frac{876001693674}{17563}x^2 + \frac{176574756573721}{210756}x - \frac{82282277487181}{17563} \\ S_5 = -\frac{60287356807}{210756}x^3 + \frac{531978629109}{35126}x^2 - \frac{56391409104893}{210756}x + \frac{55459045944329}{35126} \\ S_6 = \frac{84404065913}{210756}x^3 - \frac{770244175371}{35126}x^2 + \frac{12035521968421}{30108}x - \frac{85181016939511}{35126} \\ S_7 = -\frac{146870732089}{210756}x^3 + \frac{713433202824}{17563}x^2 - \frac{166221952457219}{210756}x + \frac{12800330212603}{2509} \\ S_8 = \frac{153363422915}{210756}x^3 - \frac{787737572196}{17563}x^2 + \frac{194059033547581}{210756}x - \frac{15793398835397}{2509} \\ S_9 = -\frac{101834083807}{210756}x^3 + \frac{1104098676189}{35126}x^2 - \frac{11043635988125}{16212}x + \frac{24684252561407}{5018} \\ S_{10} = \frac{63936334673}{210756}x^3 - \frac{719375927091}{35126}x^2 + \frac{97131379787335}{210756}x - \frac{9338266774307}{2702} \\ S_{11} = -\frac{31783843297}{210756}x^3 + \frac{190703059782}{17563}x^2 - \frac{54776542651055}{210756}x + \frac{36353549747087}{17563} \end{cases}$$

C.7. Piecewise polynomial found by Linear Spline Interpolation:

$$S(x) = \begin{cases} S_0 = 425378x + 1764096 \\ S_1 = 1274208x + 915266 \\ S_2 = 1001328x + 1461026 \\ S_3 = 11203733 - 2246241x \\ S_4 = 407361x + 589325 \\ S_5 = 8229870 - 1120748x \\ S_6 = 474471x - 1341444 \\ S_7 = 4386201 - 343764x \\ S_8 = 999916x - 6363239 \\ S_9 = 7861702 - 580633x \\ S_{10} = 384508x - 1789708 \\ S_{11} = 1838953x - 17788603 \\ S_{12} = 25306757 - 1752327x \\ S_{13} = 1084810x - 11576024 \\ S_{14} = 43765360 - 2868146x \\ S_{15} = 1179371x - 16947395 \\ S_{16} = 5392509 - 216873x \\ S_{17} = 8156845 - 379481x \\ S_{18} = 6670729 - 296919x \\ S_{19} = 404644x - 6658968 \\ S_{20} = 12496532 - 553131x \\ S_{21} = 219763x - 3734242 \\ S_{22} = 90967x - 900730 \\ S_{23} = 541644x - 11266301 \end{cases}$$

C.8. Pre Scandal Cubic Regression Polynomial:

$$P_3(x) = 16727.23630536x^3 - 281160.85389611x^2 + 1167936.16978859x + 1780431.0082417484$$

C.9. Post Scandal Quadratic Regression Polynomial:

$$P_2(x) = 43346.93906094x^2 - 1743088.46553446x + 18530741.746253766$$

C.10. Pre Scandal Linear Regression Binomial:

$$P(x) = 18728.35164835x + 2445981.813186813$$

C.11. Post Scandal Linear Regression Binomial:

$$P(x) = -182598.65934066x + 5093190.6373626385$$